

✓ Question 1: Loan Eligibility Calculator (Functions)

Description

Create a loan eligibility calculator where different functions handle income, credit score, and age to determine eligibility.

Requirements

Create separate functions for:

- `check_income(income)`: Validates if income is above minimum threshold (Rs 25,000/month)
- `check_credit_score(score)`: Validates if credit score is above 650
- `check_age(age)`: Validates if age is between 21 and 65
- `calculate_loan_amount(income, credit_score)`: Calculates eligible loan amount based on:
 - $\text{Income} \times 60$ (if credit score ≥ 750)
 - $\text{Income} \times 50$ (if credit score 700-749)
 - $\text{Income} \times 40$ (if credit score 650-699)
- `determine_eligibility(income, credit_score, age)`: Main function that uses all above functions to determine final eligibility. Display detailed eligibility report

Input/Output Example

Enter monthly income: 50000

Enter credit score: 720

Enter age: 30

=== Loan Eligibility Report ===

Income Check: ✓ Passed (Rs 50,000)

Credit Score Check: ✓ Passed (720)

Age Check: ✓ Passed (30 years)

Eligible Loan Amount: Rs 25,00,000

Status: ELIGIBLE

Constraints

- All functions must be separate and reusable
- Handle invalid inputs (negative values, out of range)
- Return appropriate boolean or numeric values

```
def check_income(income):
    return income >= 25000

def check_credit_score(score):
    return score >= 650

def check_age(age):
    return 21 <= age <= 65

def calculate_loan_amount(income, credit_score):
    if credit_score >= 750:
        return income * 60

    elif credit_score >= 700:
        return income * 50

    elif credit_score >= 650:
        return income * 40

    else:
        return 0

def determine_eligibility(income, credit_score, age):

    income_status = check_income(income)
    credit_status = check_credit_score(credit_score)
    age_status = check_age(age)

    if income_status and credit_status and age_status:
        loan_amount = calculate_loan_amount(
            income,
            credit_score
        )
        return True, loan_amount
    return False, 0

try:

    income = float(input("Enter monthly income: "))
    credit_score = int(input("Enter credit score: "))
    age = int(input("Enter age: "))
    if income < 0 or credit_score < 0 or age < 0:
        print("Error: Values cannot be negative.")

    else:

        eligible, loan_amount = determine_eligibility(
            income,
            credit_score,
            age
        )
```

```

print("\n=== Loan Eligibility Report ===")
if check_income(income):
    print(f"Income Check: ✓ Passed (Rs {income:,.0f})")
else:
    print(f"Income Check: ✗ Failed (Rs {income:,.0f})")

if check_credit_score(credit_score):
    print(f"Credit Score Check: ✓ Passed ({credit_score})")
else:
    print(f"Credit Score Check: ✗ Failed ({credit_score})")

if check_age(age):
    print(f"Age Check: ✓ Passed ({age} years)")
else:
    print(f"Age Check: ✗ Failed ({age} years)")

if eligible:
    print(f"\nEligible Loan Amount: Rs {loan_amount:,.0f}")
    print("Status: ELIGIBLE")
else:
    print("\nEligible Loan Amount: Rs 0")
    print("Status: NOT ELIGIBLE")

except ValueError:
    print("Error: Please enter valid numeric values.")

```

```

Enter monthly income: 50000
Enter credit score: 720
Enter age: 30

```

```

=== Loan Eligibility Report ===
Income Check: ✓ Passed (Rs 50,000)
Credit Score Check: ✓ Passed (720)
Age Check: ✓ Passed (30 years)

```

```

Eligible Loan Amount: Rs 2,500,000
Status: ELIGIBLE

```

✓ Question 2: Function Parameters – Flexible Calculator

Description Create a calculator that demonstrates different types of function parameters:

default arguments,

args, and *kwargs.

Requirements

Create a function calculate() that:

- Accepts a required operation parameter ('add', 'subtract', 'multiply', 'divide')
- Uses args to accept variable number of numbers
- Uses *kwargs for optional parameters: • round_to: Number of decimal places (default: 2) • display: Whether to display result (default: True)

Input/Output Example

Example 1: Using *args

```
calculate('add', 10, 20, 30, 40)
```

Output: Sum: 100.00

Example 2: Using default and **kwargs

```
calculate('multiply', 5, 6, 7, round_to=0, display=True)
```

Output: Product: 210

Constraints

- Handle division by zero
- Validate operation types
- Use appropriate parameter order (required, args, *kwargs)

```
def calculate(operation, *args, **kwargs):
    round_to = kwargs.get("round_to", 2)
    display = kwargs.get("display", True)

    if len(args) == 0:
        print("Error: No numbers provided.")
        return

    if operation == "add":
        result = sum(args)
        label = "Sum"

    elif operation == "subtract":
        result = args[0]
        for num in args[1:]:
            result -= num
        label = "Difference"

    elif operation == "multiply":
        result = 1
        for num in args:
            result *= num
        label = "Product"

    elif operation == "divide":
```

```
result = args[0]
for num in args[1:]:
    if num == 0:
        print("Error: Division by zero.")
        return
    result /= num
label = "Quotient"

else:
    print("Invalid operation.")
    return

result = round(result, round_to)

if display:
    print(f"{label}: {result}")

return result
```

```
calculate("add", 10, 20, 30, 40)
```

```
Sum: 100
100
```

```
calculate('multiply', 5, 6, 7, round_to=0, display=True)
```

```
Product: 210
210
```

› Question 3: Nested Functions - Invoice Calculator

↳ 3 cells hidden

› Question 4: Functions Returning Functions - Currency Converter Factory

↳ 2 cells hidden

› Question 5: Lambda Functions - Data Processing

↳ 2 cells hidden

› Question 6: Decorators - Function Enhancement

↳ 2 cells hidden

› Question 7: Custom Module Creation

↳ 5 cells hidden

› Question 8: Package Creation and Organization

↳ 10 cells hidden